

30.4 Componentwise boosting

Boosting is an algorithm, not a criterion: it improves the fit by many small steps, each changing the single coefficient that helps most, and the number of steps plays the part of a penalty. Begun as a way to combine classifiers (Freund and Schapire, 1997) and recast as gradient descent on a loss (Friedman, Hastie and Tibshirani, 2000; Friedman, 2001), with squared error and single regressors it is the L_2 boosting of Bühlmann and Yu (2003).

30.4.1 The algorithm

Let $\mathbf{y}$ and the columns of $\mathbf{X}$ be centred, and let no column be zero. For a vector $\mathbf{r}$, the least squares fit of $\mathbf{r}$ on the single column $\mathbf{x}_j$ has coefficient $\mathbf{x}_j^\top \mathbf{r} / \|\mathbf{x}_j\|^2$. It reduces the sum of squares by $(\mathbf{x}_j^\top \mathbf{r})^2 / \|\mathbf{x}_j\|^2$, by the one-column case of Theorem 6.4.1.

Definition 30.4.1 — Componentwise L_2 boosting

Fix a **step size** $\nu \in (0, 2)$. Start from $\mathbf{b}^{(0)} = \mathbf{0}$ and $\mathbf{r}^{(0)} = \mathbf{y}$. For $m = 0, 1, 2, \dots$:

1. choose $j_m = \arg \max_j (\mathbf{x}_j^\top \mathbf{r}^{(m)})^2 / \|\mathbf{x}_j\|^2$, the column whose simple regression fits the current residual best;
2. set $b_{j_m}^{(m+1)} = b_{j_m}^{(m)} + \nu \mathbf{x}_{j_m}^\top \mathbf{r}^{(m)} / \|\mathbf{x}_{j_m}\|^2$, leave the other coordinates unchanged, and put $\mathbf{r}^{(m+1)} = \mathbf{y} - \mathbf{X}\mathbf{b}^{(m+1)}$.

The estimate after m steps is $\mathbf{b}^{(m)}$, and the **stopping iteration** m is the tuning parameter.

With unit-length columns, step 1 picks the column most correlated with the residual. The residual is the negative gradient of $\frac{1}{2}\|\mathbf{y} - \mathbf{f}\|^2$ in the fitted vector $\mathbf{f}$, so each step fits the negative gradient by the best single column and moves a fraction ν of the way. The theory below allows $\nu < 2$; with $\nu > 1$ a step overshoots the simple regression fit, and in practice $\nu \leq 1$, often $\nu = 0.1$. With $\nu = 1$ this is *matching pursuit* (Mallat and Zhang, 1993), which is not forward selection, since earlier coefficients are never refitted.

30.4.2 Convergence to least squares

Boosting converges to the least squares fit at a geometric rate, slow when the design is badly conditioned.

Theorem 30.4.1 — Convergence of componentwise L_2 boosting

Let $\nu \in (0, 2)$, let $Q(\mathbf{b}) = \frac{1}{2}\|\mathbf{y} - \mathbf{X}\mathbf{b}\|^2$ with minimum Q^* , let $s > 0$ be the smallest nonzero singular value of $\mathbf{X}$, and put

$$\kappa = \frac{\nu(2 - \nu)s^2}{p \max_j \|\mathbf{x}_j\|^2}.$$

Then $0 < \kappa \leq 1$, and:

- (a) each step lowers the residual sum of squares by exactly

$$Q(\mathbf{b}^{(m)}) - Q(\mathbf{b}^{(m+1)}) = \nu \left(1 - \frac{\nu}{2}\right) \max_j \frac{(\mathbf{x}_j^\top \mathbf{r}^{(m)})^2}{\|\mathbf{x}_j\|^2}; \quad (30.4.1)$$

- (b) $Q(\mathbf{b}^{(m)}) - Q^* \leq (1 - \kappa)^m (Q(\mathbf{0}) - Q^*)$. Hence the fitted values converge geometrically to the least squares fit: $\|\mathbf{X}\mathbf{b}^{(m)} - \mathbf{M}\mathbf{y}\|^2 = 2\{Q(\mathbf{b}^{(m)}) - Q^*\} \rightarrow 0$;
(c) the coefficients $\mathbf{b}^{(m)}$ converge to a least squares estimate. If $\mathbf{X}$ has full column rank, the limit is $\hat{\boldsymbol{\beta}}$.

Proof.

- (a) Write $\mathbf{r} = \mathbf{r}^{(m)}$, $j = j_m$ and $c = \mathbf{x}_j^\top \mathbf{r} / \|\mathbf{x}_j\|^2$. Then $\mathbf{r}^{(m+1)} = \mathbf{r} - \nu c \mathbf{x}_j$, and

$$\begin{aligned} \|\mathbf{r} - \nu c \mathbf{x}_j\|^2 &= \|\mathbf{r}\|^2 - 2\nu c \mathbf{x}_j^\top \mathbf{r} + \nu^2 c^2 \|\mathbf{x}_j\|^2 \\ &= \|\mathbf{r}\|^2 - (2\nu - \nu^2) \frac{(\mathbf{x}_j^\top \mathbf{r})^2}{\|\mathbf{x}_j\|^2}. \end{aligned}$$

Halving gives Eq. 30.4.1, since j maximizes the ratio.

- (b) Let $\mathbf{u}_m = \mathbf{M}\mathbf{r}^{(m)} = \mathbf{M}\mathbf{y} - \mathbf{X}\mathbf{b}^{(m)}$, which lies in $\mathcal{C}(\mathbf{X})$. The residual splits orthogonally as $\mathbf{r}^{(m)} = (\mathbf{I} - \mathbf{M})\mathbf{y} + \mathbf{u}_m$, so $Q(\mathbf{b}^{(m)}) - Q^* = \frac{1}{2}\|\mathbf{u}_m\|^2$ and $\mathbf{X}^\top \mathbf{r}^{(m)} = \mathbf{X}^\top \mathbf{u}_m$. By the singular value decomposition (Theorem 1.8.1), $\|\mathbf{X}^\top \mathbf{u}\| \geq s\|\mathbf{u}\|$ for every $\mathbf{u} \in \mathcal{C}(\mathbf{X})$. A maximum is at least an average, so

$$\begin{aligned} \max_j \frac{(\mathbf{x}_j^\top \mathbf{r}^{(m)})^2}{\|\mathbf{x}_j\|^2} &\geq \frac{\sum_j (\mathbf{x}_j^\top \mathbf{u}_m)^2}{p \max_j \|\mathbf{x}_j\|^2} \\ &= \frac{\|\mathbf{X}^\top \mathbf{u}_m\|^2}{p \max_j \|\mathbf{x}_j\|^2} \geq \frac{s^2 \|\mathbf{u}_m\|^2}{p \max_j \|\mathbf{x}_j\|^2}. \end{aligned}$$

With (a), $Q(\mathbf{b}^{(m)}) - Q(\mathbf{b}^{(m+1)}) \geq \kappa\{Q(\mathbf{b}^{(m)}) - Q^*\}$, that is, $Q(\mathbf{b}^{(m+1)}) - Q^* \leq (1 - \kappa)\{Q(\mathbf{b}^{(m)}) - Q^*\}$. Iterating gives (b). Finally $\kappa > 0$, and $\kappa \leq 1$ because $\nu(2 - \nu) \leq 1$ and $s^2 \leq \text{tr}(\mathbf{X}^\top \mathbf{X}) = \sum_j \|\mathbf{x}_j\|^2 \leq p \max_j \|\mathbf{x}_j\|^2$.

- (c) The m th step changes one coordinate, by $\nu |\mathbf{x}_{j_m}^\top \mathbf{u}_m| / \|\mathbf{x}_{j_m}\|^2 \leq \nu \|\mathbf{x}_{j_m}\| \|\mathbf{u}_m\| / \|\mathbf{x}_{j_m}\|^2 \leq \nu \|\mathbf{u}_m\| / \min_j \|\mathbf{x}_j\|$, using Cauchy–Schwarz. By (b), $\|\mathbf{u}_m\| \leq (1 - \kappa)^{m/2} \|\mathbf{u}_0\|$, which is summable. So $\sum_m \|\mathbf{b}^{(m+1)} - \mathbf{b}^{(m)}\| < \infty$, the sequence is Cauchy, and it converges to some $\mathbf{b}^\infty$. By continuity, $\mathbf{X}\mathbf{b}^\infty = \lim \mathbf{X}\mathbf{b}^{(m)} = \mathbf{M}\mathbf{y}$, so $\mathbf{b}^\infty$ is a least squares estimate (Theorem 6.4.1). With full column rank there is only one. ■

The bound is crude, but it shows what controls the speed: the step size and the conditioning of $\mathbf{X}$.

Remark (When $p > n$). If $\text{rank } \mathbf{X} = n$, as is typical when $p > n$, boosting run long enough *interpolates* the data, so stopping early is a necessity. Its limit depends on the order in which columns were chosen and is in general not the least squares estimate of smallest norm (Proposition 6.10.1), which full gradient descent from zero does reach (Exercise (B2); this is the ridgeless estimator of Theorem 28.1.1).

30.4.3 Early stopping as shrinkage

For a base learner linear in the data, the effect of stopping early is exact. Let an $n \times n$ matrix $\mathbf{S}$ map a residual vector to its fitted values; boosting updates $\mathbf{f}^{(m+1)} = \mathbf{f}^{(m)} + \nu \mathbf{S}(\mathbf{y} - \mathbf{f}^{(m)})$ from $\mathbf{f}^{(0)} = \mathbf{0}$.

Proposition 30.4.2 — The boosting operator

- (a) $\mathbf{f}^{(m)} = \mathbf{B}_m \mathbf{y}$ with $\mathbf{B}_m = \mathbf{I} - (\mathbf{I} - \nu \mathbf{S})^m$.
- (b) Let $\mathbf{S}$ be symmetric with eigenvalues $d_k \geq 0$ and orthonormal eigenvectors $\mathbf{u}_k$, and let $\nu d_k \leq 1$ for every k . Then

$$\mathbf{B}_m = \sum_k \{1 - (1 - \nu d_k)^m\} \mathbf{u}_k \mathbf{u}_k^\top.$$

Each factor $1 - (1 - \nu d_k)^m$ lies in $[0, 1]$, is nondecreasing in m , and tends to 1 when $d_k > 0$. The trace $\text{tr } \mathbf{B}_m$ increases with m towards $\text{rank } \mathbf{S}$.

- (c) (*Least squares learner.*) If $\mathbf{S} = \mathbf{M}$, then $\mathbf{B}_m = \{1 - (1 - \nu)^m\} \mathbf{M}$. Early stopping shrinks the least squares fit by a single factor.
- (d) (*Gradient descent.*) The iteration $\mathbf{b}^{(m+1)} = \mathbf{b}^{(m)} + \nu \mathbf{X}^\top (\mathbf{y} - \mathbf{X} \mathbf{b}^{(m)})$ from $\mathbf{b}^{(0)} = \mathbf{0}$ is the case $\mathbf{S} = \mathbf{X} \mathbf{X}^\top$. If $\mathbf{X}$ has singular values σ_k and $\nu \sigma_k^2 \leq 1$, its fitted values shrink the k th singular component of the least squares fit by $1 - (1 - \nu \sigma_k^2)^m$. Ridge regression shrinks the same component by $\sigma_k^2 / (\sigma_k^2 + \lambda)$.

Proof.

- (a) $\mathbf{y} - \mathbf{f}^{(m+1)} = (\mathbf{I} - \nu \mathbf{S})(\mathbf{y} - \mathbf{f}^{(m)})$, so $\mathbf{y} - \mathbf{f}^{(m)} = (\mathbf{I} - \nu \mathbf{S})^m \mathbf{y}$. (b) $(\mathbf{I} - \nu \mathbf{S})^m = \sum_k (1 - \nu d_k)^m \mathbf{u}_k \mathbf{u}_k^\top$ by the spectral theorem (Theorem 1.7.2), and $0 \leq 1 - \nu d_k \leq 1$.
- (c) $\mathbf{M}$ has eigenvalues 0 and 1, and it is idempotent.
- (b) $\mathbf{X} \mathbf{b}^{(m+1)} = \mathbf{X} \mathbf{b}^{(m)} + \nu \mathbf{X} \mathbf{X}^\top (\mathbf{y} - \mathbf{X} \mathbf{b}^{(m)})$. The eigenvalues of $\mathbf{X} \mathbf{X}^\top$ are the σ_k^2 and zeros, and the eigenvectors for the σ_k^2 are the left singular vectors (Theorem 1.8.1). For ridge, see Theorem 27.2.2.

■

For small σ_k^2 , gradient descent keeps the fraction $1 - (1 - \nu \sigma_k^2)^m \approx m \nu \sigma_k^2$ of a component and ridge keeps $\approx \sigma_k^2 / \lambda$; for large σ_k^2 both keep nearly everything. So m steps behave like ridge with $\lambda \approx 1/(m \nu)$. Part (c) is the proportional shrinkage of Proposition 7.6.3(c).

Degrees of freedom. For a linear smoother the complexity is $\text{tr } \mathbf{B}_m$, which equals the covariance penalty $\sigma^{-2} \sum_i \text{Cov}(\hat{y}_i, y_i)$ (Theorem 29.1.1). Componentwise boosting is not linear, since the chosen columns depend on $\mathbf{y}$, but for a fixed sequence $j_0, \dots, j_{m-1}$ its fitted values are $\mathbf{B}_m \mathbf{y}$ with

$$\mathbf{B}_m = \mathbf{I} - (\mathbf{I} - \nu \mathbf{H}_{j_{m-1}}) \cdots (\mathbf{I} - \nu \mathbf{H}_{j_0}), \quad \mathbf{H}_j = \frac{\mathbf{x}_j \mathbf{x}_j^\top}{\|\mathbf{x}_j\|^2}, \quad (30.4.2)$$

by Proposition 30.4.2(a) applied step by step. Bühlmann (2006) used $\text{tr } \mathbf{B}_m$ along the observed sequence as the degrees of freedom in a corrected Akaike criterion for choosing m . That treats

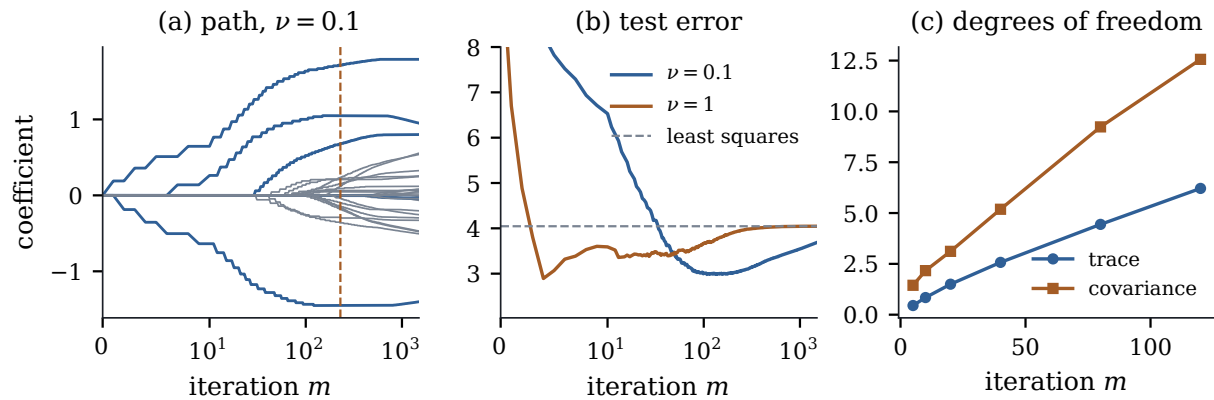


Figure 30.4.1: Componentwise boosting for [Example \(Boosting a sparse regression\)](#). (a) Coefficient paths with $\nu = 0.1$ (true nonzero coefficients in blue); the dashed line is the cross-validated stopping iteration. (b) Exact risk along the path for $\nu = 0.1$ and $\nu = 1$, with the least squares risk dashed. Both horizontal axes are linear up to 10 and logarithmic beyond. (c) Two degrees of freedom: the average trace of the boosting operator [Eq. 30.4.2](#), and the covariance degrees of freedom estimated from 2000 simulated responses.

the selection as fixed in advance, and the next example shows how much it can understate.

Example. (Boosting a sparse regression)

The script draws $n = 60$ observations of $p = 30$ normal regressors with correlations $0.5^{|j-k|}$, five nonzero coefficients and $\sigma = 1.5$, and boosts on the standardized regressors. The expected squared error on a new case (the *risk*) is computed exactly from the design law; its floor is $\sigma^2 = 2.25$.

Convergence. For $\nu = 0.1$, $\kappa = 3.18 \times 10^{-4}$, and $\|\mathbf{b}^{(m)} - \hat{\boldsymbol{\beta}}\|/\|\hat{\boldsymbol{\beta}}\|$ is 0.497, 0.161 and 2.7×10^{-8} after 100, 1000 and 50000 steps; the script checks [Eq. 30.4.1](#) and the bound (b) at every step.

Early stopping. Least squares has risk 4.046. With $\nu = 0.1$ the risk is smallest, 2.981, after 108 steps (with 14 nonzero coefficients), and within 5 per cent of that from $m = 72$ to $m = 305$; five-fold cross-validation chooses $m = 229$, with risk 3.047. With $\nu = 1$ the best risk, 2.893, occurs at $m = 4$ and only there is the risk within 5 per cent of it: a needle that a data-based stopping rule is unlikely to hit ([Figure 30.4.1](#)).

Degrees of freedom. Over 2000 responses with the same design and mean, the covariance degrees of freedom after $m = 20$ steps are 3.1 against an average trace of 1.5, and after $m = 120$ steps 12.6 against 6.2. The trace does not see the adaptation of the chosen columns to the noise.

```
import numpy as np

def l2_boost(X, y, nu, steps):
    """Componentwise L2 boosting from b = 0; returns the coefficient path (steps +
    1) x p."""
    n, p = X.shape
    norms2 = np.sum(X ** 2, axis=0)
    b = np.zeros(p)
    r = y.astype(float).copy()
    path = [b.copy()]
```

```

for _ in range(steps):
    corr = X.T @ r
    j = np.argmax(corr ** 2 / norms2)          # largest drop in the
    residual sum of squares
    step = nu * corr[j] / norms2[j]
    b[j] += step
    r -= step * X[:, j]
    path.append(b.copy())
return np.array(path)

rng = np.random.default_rng(3004)
n, p, sigma = 60, 30, 1.5
C = 0.5 ** np.abs(np.subtract.outer(np.arange(p), np.arange(p)))
L = np.linalg.cholesky(C)
beta = np.zeros(p)
beta[[0, 4, 9, 14, 19]] = [2.0, -1.5, 1.0, 1.0, -0.5]

def draw(m):
    Z = rng.normal(size=(m, p)) @ L.T
    return Z, Z @ beta + sigma * rng.normal(size=m)

X, y = draw(n)
mx, sx, my = X.mean(axis=0), X.std(axis=0), y.mean()
Xs, ys = (X - mx) / sx, y - my                # centre and scale by the
training data

def risk(path):
    """Expected squared error on a new case, for each row b of path (exact, from the
    design law)."""
    d = beta[None, :] - path / sx              # coefficient error on the
    original scale
    offset = path @ (mx / sx) - my             # error in the fitted
    intercept
    return np.sum((d @ C) * d, axis=1) + offset ** 2 + sigma ** 2

b_ls = np.linalg.lstsq(Xs, ys, rcond=None)[0]
M = 1500
test_err = {}
for step in [0.1, 1.0]:
    test_err[step] = risk(l2_boost(Xs, ys, step, M))
err_ls = risk(b_ls[None, :])[0]
m_best = int(np.argmin(test_err[0.1]))
print(f"least squares: risk {err_ls:.3f}")
print(f"nu = 0.1: best m = {m_best}, risk {test_err[0.1][m_best]:.3f}")
print(f"nu = 1.0: best m = {np.argmin(test_err[1.0])}, risk {test_err[1.0].min():.3f}
    ")

```

[Run this code online](#)

The next cell repeats the comparison with 300 responses and $m = 20$; the script's run of it gives 3.2 against 1.5.

```

mu = Xs @ (beta * sx)                        # the true mean, in the
centred scale
def operator_trace(X, path, nu):
    """Trace of  $B_m = I - \text{prod}_k (I - \nu H_{\{j_k\}})$ , with the selected columns held
    fixed."""
    n = X.shape[0]

```

```

Rm = np.eye(n)
for k in range(len(path) - 1):
    j = int(np.flatnonzero(path[k + 1] != path[k])[0])
    x = X[:, j]
    Rm -= nu * np.outer(x, x @ Rm) / (x @ x)
return n - np.trace(Rm)

reps, m_df = 300, 20
rng_df = np.random.default_rng(30041)
Y = mu + sigma * rng_df.normal(size=(reps, n))          # repeated responses, same
                  design and mean
paths = [l2_boost(Xs, Y[r], 0.1, m_df) for r in range(reps)]
F = np.array([Xs @ P[-1] for P in paths])              # fitted values after m_df
                  steps
cov_df = np.sum(np.mean((Y - Y.mean(axis=0)) * (F - F.mean(axis=0)), axis=0)) /
          sigma ** 2
trace_df = np.mean([operator_trace(Xs, P, 0.1) for P in paths])
print(f"m = {m_df}: mean trace {trace_df:.1f}, covariance df {cov_df:.1f}")

```

[Run this code online](#)

The gap is the optimism of selection (Proposition 29.5.1). For the lasso, by contrast, the covariance degrees of freedom equal the expected number of nonzero coefficients (Zou, Hastie and Tibshirani, 2007; we do not reproduce the proof): shrinkage offsets the cost of selection (compare [Exercise \(C1\)](#)). In practice m is chosen by cross-validation ([Chapter 29](#)), which needs no degrees of freedom.

30.4.4 Exercises

30.4.4.1 A. Check your understanding

Exercise A1.

Let $\mathbf{X}^\top \mathbf{X} = \mathbf{I}$, $\mathbf{z} = \mathbf{X}^\top \mathbf{y} = (3, -1, 0.5)^\top$ and $\nu = 0.1$. Carry out the first three boosting steps by hand. Show that under an orthonormal design the current residual correlations are $\mathbf{z} - \mathbf{b}^{(m)}$, so boosting always moves the coordinate whose remaining correlation is largest in absolute value, and removes one tenth of that correlation.

Solution. $\mathbf{X}^\top \mathbf{r}^{(m)} = \mathbf{X}^\top \mathbf{y} - \mathbf{X}^\top \mathbf{X} \mathbf{b}^{(m)} = \mathbf{z} - \mathbf{b}^{(m)}$, with unit column lengths. Step 1: coordinate 1, $b_1 = 0.3$, residual correlation 2.7. Step 2: coordinate 1 again, $b_1 = 0.3 + 0.27 = 0.57$, correlation 2.43. Step 3: coordinate 1, $b_1 = 0.57 + 0.243 = 0.813$. Coordinate 1 keeps being chosen until its remaining correlation $3(0.9)^m$ falls below 1, which first happens at $m = 11$. This is the approach to soft thresholding that [Section 30.5](#) makes precise.

30.4.4.2 B. Practice

Exercise B1.

Use Eq. 30.4.1 to show that $\sum_{m \geq 0} \max_j (\mathbf{x}_j^\top \mathbf{r}^{(m)})^2 / \|\mathbf{x}_j\|^2 \leq \|\mathbf{y}\|^2 / \{\nu(2 - \nu)\}$. Deduce, without the singular value argument, that $\mathbf{X}^\top \mathbf{r}^{(m)} \rightarrow \mathbf{0}$ for every $\nu \in (0, 2)$.

Solution. Summing Eq. 30.4.1 over $m = 0, \dots, M-1$ gives $\nu(1 - \nu/2) \sum_{m < M} \max_j (\dots) = Q(\mathbf{0}) - Q(\mathbf{b}^{(M)}) \leq \frac{1}{2} \|\mathbf{y}\|^2$. Multiply by $2/\{\nu(2 - \nu)\}$ and let $M \rightarrow \infty$. The terms of a convergent series tend to zero. So $\max_j |\mathbf{x}_j^\top \mathbf{r}^{(m)}| / \|\mathbf{x}_j\| \rightarrow 0$, and hence every coordinate of $\mathbf{X}^\top \mathbf{r}^{(m)}$ does.

Exercise B2.

For the gradient descent iteration of Proposition 30.4.2(d) with $0 < \nu < 2/\sigma_1^2$, show that every $\mathbf{b}^{(m)}$ lies in $\mathcal{C}(\mathbf{X}^\top)$, and that $\mathbf{b}^{(m)} \rightarrow \mathbf{X}^+ \mathbf{y}$, the minimum-norm least squares estimate (Proposition 6.10.1). Why does this argument fail for componentwise boosting?

Solution. Each increment $\nu \mathbf{X}^\top (\mathbf{y} - \mathbf{X} \mathbf{b}^{(m)})$ lies in $\mathcal{C}(\mathbf{X}^\top)$, and $\mathbf{b}^{(0)} = \mathbf{0}$. In the singular value basis, $\mathbf{b}^{(m)} = \sum_k \{1 - (1 - \nu \sigma_k^2)^m\} \sigma_k^{-1} (\mathbf{u}_k^\top \mathbf{y}) \mathbf{v}_k$, because the iteration acts on each component separately. With $|1 - \nu \sigma_k^2| < 1$ this converges to $\sum_k \sigma_k^{-1} (\mathbf{u}_k^\top \mathbf{y}) \mathbf{v}_k = \mathbf{X}^+ \mathbf{y}$. Componentwise boosting adds multiples of coordinate vectors $\mathbf{e}_j$, which need not lie in $\mathcal{C}(\mathbf{X}^\top)$ when $\text{rank } \mathbf{X} < p$. So its limit can have a component in $\mathcal{N}(\mathbf{X})$.

30.4.4.3 C. Going deeper

Exercise C1.

Under an orthonormal design with $\mathbf{y} \sim N_n(\mathbf{X}\boldsymbol{\beta}, \sigma^2 \mathbf{I})$, the lasso fit with parameter λ has covariance degrees of freedom $E \#\{j : |z_j| > \lambda\}$ (by Stein's lemma, Lemma 27.5.1, applied coordinatewise). Show that hard thresholding at the same λ has covariance degrees of freedom $E \#\{j : |z_j| > \lambda\} + \sum_j (\lambda/\sigma) \{\phi((\lambda - \beta_j)/\sigma) + \phi((\lambda + \beta_j)/\sigma)\}$, where ϕ is the standard normal density. The extra term is the price of selecting without shrinking.